INF_Datenaustauschformate

Website:	mebis | Lernplattform	Gedruckt von:	Ioannis Kiourtzidis
Kurs:	AEuP11		
Buch:	INF_Datenaustauschformate	Datum:	Dienstag, 8. Oktober 2024, 12:33

Inhaltsverzeichnis

Datenaustauschformate

Datenaustauschformate

Übersicht

Datenquellen stellen für eine Anwendung die verfügbaren Daten bereit, mit denen die Anwendung arbeiten möchte. Dabei werden interne und externe Datenquellen unterschieden. Interne Quellen sind z. B. Produktionsdaten, Artikeldaten oder andere

operative Anwendungssysteme. Externe Datenquellen sind beispielsweise öffentliche Datenbanken wie Wetter-, Bevölkerungs-, Marktforschungsdaten.

Werden Daten verarbeitet, dann reserviert das verarbeitende Programm Platz im Arbeitsspeicher. Dabei werden die Daten nach einem eigenen System abgelegt. Solange das Programm läuft, kann auf die Daten zugegriffen werden. Das geht aber nicht mehr, sobald die Daten auch in anderen Programmen verwendet werden sollen. Das ist zum Beispiel der Fall, wenn das Programm die Daten auf der Festplatte speichern und später wieder einlesen soll oder wenn es Daten an einen Server schicken muss.

Sowohl die internen als auch externen Datenquellen stehen in unterschiedlichen Dateiformaten zur Verfügung. Für die Erfassung, Speicherung und den Austausch von unstrukturierten oder teilweise strukturierten Daten können sowohl proprietäre als auch standardisierte Datenformate verwendet werden.

CSV, XML und JSON sind wahrscheinlich die am häufigsten verwendeten Datenaustauschformate. Sie sind unabhängig von einer bestimmten Programmiersprache und Umgebung und die Inhalte sind sowohl für Menschen als auch Maschinen lesbar.

CSV




```
Vorname,Nachname,Geburtsdatum,Straße,Stadt
Ludwig,Erhardt,1897-02-04,Sternstr. 5,Fürth
Martin,Segitz,1853-07-26,,Fürth
…
```

Dateien vom Typ CSV (Comma-Separated Values, durch Trennzeichen getrennte Werte) werden häufig verwendet, um tabellarische strukturierte Daten zwischen Systemen auszutauschen. CSV Dateien werden dabei von einer Vielzahl geschäftlicher, verbraucherorientierter und wissenschaftlicher Anwendungen unterstützt. So können CSV-Dateien beispielsweise von Datenbank- und Tabellenkalkulationsprogrammen importiert und exportiert werden.

Aufbau

- Die Trennung von Datensätzen erfolgt in der Regel über einen Zeilenumbruch
- Ein Trennzeichen (z. B. Kommata, Semikolon) wird zur Trennung von Datenfeldern (Spalten) innerhalb der Datensätze benutzt. Da Kommata oftmals Inhalt eines Tabellenwertes sind, wird an dieser Stelle die Verwendung des Semikolons als Trennzeichen empfohlen.

- Um Sonderzeichen innerhalb der Daten nutzen zu können (z. B. Kommata in Dezimalzahlwerten), wird ein Feldbegrenzerzeichen benutzt. Normalerweise ist dieser Feldbegrenzer das Anführungszeichen ".
- Der erste Datensatz einer CSV ist typischerweise ein Kopfdatensatz, der die Spaltennamen (Überschriften) definiert.
- Jede CSV sollte nur eine Tabelle enthalten. Für Datenbeziehungen werden üblicherweise mehrere CSV-Dateien verwendet.

XML

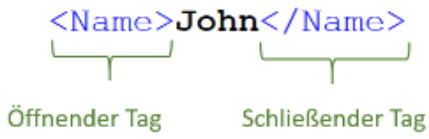


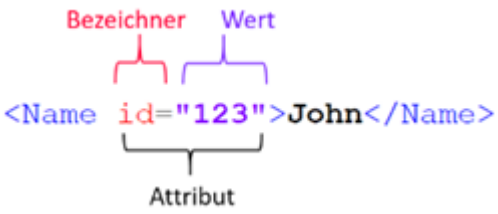

```
<?xml version="1.0" encoding="UTF-8"?>
<schema  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <element name="Personen">
    <complexType>
      <sequence>
        <element type="string" name="Vorname"/>
        <element type="string" name="Nachname"/>
        <element type="date" name="Geburtsdatum"/>
        ...
      </sequence>
    </complexType>
  </element>
```

XML (eXtensible Markup Language, dt. *Erweiterbare Auszeichnungssprache*) ist eine Auszeichnungssprache zur Darstellung hierarchischer strukturierter Daten. Sie ist eine Metasprache. Das bedeutet, dass auf Basis von XML anwendungsspezifische Sprachen (RSS, XHTML, SVG, ...) definiert werden.

Aufbau

- Ein XML besteht aus Elementen. Elemente können sowohl Text als auch weitere Elemente beinhalten.
- Ein Dokument besitzt ein Wurzelelement.
- Für die Beschreibung von Elementen werden Tags verwendet <Elementname>.
- Elemente stehen innerhalb eines Start- und eines Endtags. </Elementname>.
- Ist ein Element leer, dann kann ein Leertag verwendet werden. <Leer />.
- Jedes Element kann Attribute haben. Ein Attribut kann ein Attributwert zugewiesen werden. Diese stehen in Anführungsstrichen.





Ein XML ist **wohlgeformt**, wenn alle XML-Regeln eingehalten werden.

XML-Schema

Durch XML-Schema kann geprüft werden, ob ein XML Dokument nicht nur wohlgeformt, sondern auch “**gültig**” ist. Die Gültigkeit wird Anhand eines Schemas vorgegeben.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<schema  xmlns:xs="http://www.w3.org/2001/XMLSchema">
  <element name="Personen">
    <complexType>
      <sequence>
        <element type="string" name="Vorname"/>
        <element type="string" name="Nachname"/>
        <element type="date" name="Geburtsdatum"/>
        ...
      </sequence>
    </complexType>
  </element>
```

Das Schema kann entweder in der Datei enthalten sein oder es wird ein externes Schema eingebunden. Externe Schema werden in einer Datei mit der Endung .dtd (**Document Type Definition**) gespeichert und zu Beginn des XMLs eingebunden.

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE kursprogramm SYSTEM "kursprogramm.dtd">
<kursprogramm>
  ...
```

> Statt einem XML Schema werden die Regeln zum Aufbau eines XMLs häufig auch in einer DTD Datei hinterlegt. Vor- und Nachteile sind zum Beispiel hier diskutiert:
> **https://stackoverflow.com/questions/1544200/what-is-difference-between-xml-schema-and-dtd**

JSON



Dateien vom Typ JSON (JavaScript Object Notation) werden häufig verwendet, um Daten zu speichern und zwischen Systemen im Nur-Text-Format auszutauschen.

```json
{
  "Vorname": "Martin",
  "Nachname": "Segitz",
  "Geburtsdatum": "1853-07-26",
  "Adresse": {
    "Straße": null,
    "Stadt": "Fürth"
  },
  "Berufe": [
    "Zinngießer",
    "Redakteur",
    "Bayerischer Landtag",
    "Deutscher Reichstag"
  ]
}
```

JSON wird dabei häufig für die Kommunikation zwischen Clients und Server verwendet. Vorteile gegenüber XML ergeben sich aus der schnelleren und einfacheren Lesbarkeit der Daten. Zudem kann sehr einfach ein JavaScript Objekt erstellt und weiterverwendet werden.

JSON ist Key-Value orientiert und basiert auf folgenden Basis-Datenstrukturen die miteinander beliebig verschachtelt werden können.

Datenstrukturen in JSON

object	{ "key1": "value1", "key2": "value2" }
array	["first", "second", "third"]
number	42 3.1415926
string	"Hallo Welt"
boolean	true false
null	null

JSON-Schema

Durch JSON Schema kann eine gewünschte Struktur vorgegeben werden. Dadurch können “gültige” Dokumente beschrieben werden. Anhand dieses Schemas werden die Daten validiert. Das JSON Schema orientiert sich dabei an dem XML-Schema. Nachdem das JSON Schema erstellt ist, kann es mit Hilfe von Validierungstools geprüft werden (Python: Bilbliothek jsonschema). Derzeit gibt es mehrere Validatoren für verschiedene Programmiersprachen.

```
{
  "type": "object",
  "title": "Personen",
  "required": ["Vorname", "Nachname"],
  "properties": {
    "Vorname": {"type": "string"},
    "Nachname": {"type": "string"},
    "Geburtsdatum": {
          "type": "string",
          "format": "date"
    },
    "Adresse": {
      "Straße": {"type": "string"},
      "Stadt": {"type": "string"}
    },
    "Berufe": {"type": "array"}
  }
}
```